HD-TSK-PINN: A Hierarchical and Difficulty-Aware TSK Fuzzy System

for Physics-Informed Neural Networks

[Authors — to be filled]

[Affiliations — to be filled]

Abstract

Physics-Informed Neural Networks (PINNs) have emerged as a powerful mesh-free framework for solving partial differential equations (PDEs), yet they suffer from three persistent failure modes: (i) scatter-point optimization on finite collocation sets fails to capture continuous PDE constraints, especially for hidden higher-order terms; (ii) conflicting gradient directions among PDE residual, boundary, and initial-condition losses impede optimization; and (iii) single global networks lack the localized expressiveness needed for spatially heterogeneous solutions. Existing fuzzy-PINN hybrids address feature uncertainty or representation fusion, but none exploits the structural strength of the Takagi–Sugeno–Kang (TSK) fuzzy system—rule-based soft partitioning, local expert modeling, and responsibility-weighted aggregation—as an organizational backbone for PDE solving.

This paper proposes HD-TSK-PINN, in which TSK rules simultaneously serve as soft domain decomposers, difficulty detectors, and task-weight coordinators. Four tightly coupled contributions distinguish this framework from prior work. First, a two-level hierarchical TSK antecedent performs coarse-to-fine soft domain decomposition, matching the multi-scale structure of PDE solutions. Second, a shared-backbone plus rule-adapter consequent achieves local expressiveness with O(1) extra parameters per rule. Third, a region-level difficulty score is computed per rule via neighborhood Monte Carlo sampling over an ε-ball around each collocation point, producing a signal that is robust to collocation overfitting and drives adaptive resampling and rule evolution (grow, split, prune). Fourth, a gradient-conflict-aware task weighting mechanism jointly uses rule-aggregated difficulty and pairwise gradient alignment scores to balance PDE, boundary, and initial-condition losses. A rule-balance regularizer further prevents early rule collapse. Experiments on Poisson, Burgers, Allen–Cahn, and Navier–Stokes equations demonstrate that HD-TSK-PINN consistently improves accuracy and training stability over vanilla PINNs and recent fuzzy-PINN and domain-decomposition baselines.

Keywords: Physics-informed neural networks · TSK fuzzy systems · hierarchical domain decomposition · region-level difficulty · gradient conflict · adaptive collocation

1. Introduction

Partial differential equations govern the mathematical description of physical, biological, and engineering systems. Classical numerical methods—finite difference, finite element, and spectral methods—are mature but require mesh construction and can be expensive when geometry is irregular, dimensionality is high, or repeated solves across parameter families are needed [1]. Physics-Informed Neural Networks (PINNs), introduced by Raissi et al. [2], offer a mesh-free alternative: a neural network $u_\theta(\xi)$ is trained to simultaneously minimize PDE residuals at interior collocation points, boundary condition violations, and initial condition errors, using automatic differentiation to evaluate the governing operator.

Despite widespread application in fluid dynamics, solid mechanics, and biomedical modeling [3], vanilla PINNs suffer from three well-documented failure modes. First, point-level collocation optimization: training on a finite scatter-set of collocation points does not guarantee satisfaction of the continuous PDE; in particular, optimizing only at fixed points can miss hidden higher-order constraint violations in the surrounding neighborhood [4]. Second, gradient conflict: the PINN loss comprises heterogeneous objectives (PDE, BC, IC) whose gradients with respect to shared parameters can point in opposing directions, causing net cancellation of gradient signal and training instability [5]. Third, spectral bias and local heterogeneity: standard fully-connected networks are biased toward low-frequency functions and cannot efficiently allocate expressive capacity to localized features such as shock fronts, boundary layers, or phase interfaces [6].

Three parallel research directions have emerged to address these problems. Domain-decomposition methods such as FBPINNs [7] and APINN [8] assign local subnetworks to subregions, demonstrating that soft decomposition with parameter sharing substantially improves accuracy on multi-scale problems. Region-aware optimization, exemplified by RoPINN [4], extends point-level training to continuous neighborhood regions, providing a theoretical reduction in generalization error for hidden higher-order constraints. Adaptive loss weighting and gradient alignment methods [5, 18] address the multi-task optimization challenge by detecting and mitigating gradient conflicts.

A separate line of work has explored fuzzy-neural hybrids. FPINNs [10] incorporate fuzzy membership and rule layers primarily for uncertainty-aware representation; CAFPINN [11] adds cross-attention fusion between fuzzy and neural feature branches. These works confirm the complementarity of fuzzy and neural representations, but neither uses TSK rules as adaptive domain decomposers nor couples them to region difficulty or gradient conflict. The hierarchical, rule-based structure of TSK systems—a natural soft partition of the input space with local consequent models and responsibility-weighted aggregation—is precisely what is needed to organize the above three improvements into a coherent framework.

This paper proposes HD-TSK-PINN, which repositions TSK from a peripheral feature-fusion layer to the organizational backbone of PINN training. Our central argument is that TSK rules should simultaneously serve as (i) a hierarchical soft domain decomposer, (ii) a region-level difficulty detector that guides collocation resampling and rule evolution, and (iii) a task-weight coordinator that couples difficulty and gradient conflict. The four main contributions are:

- Hierarchical TSK antecedent: a two-level (coarse + fine) fuzzy rule structure that provides coarse-to-fine inductive bias aligned with the multi-scale structure of PDE solutions, motivated by multilevel domain decomposition theory [7].
- Shared-backbone adapter consequents: the consequent of each rule is parameterized as a global backbone network plus a small rule-specific adapter, achieving local expressiveness without parameter count scaling linearly in rule count.
- Region-level difficulty scoring and rule evolution: we compute a neighborhood-smoothed residual per collocation point via Monte Carlo sampling over an ε-ball, aggregate it by rule responsibility to obtain a region-level difficulty score, and use this score to drive adaptive PDE resampling and rule grow/split/prune evolution. This design is robust to collocation overfitting, a weakness of point-level difficulty monitoring.
- Gradient-conflict-aware task weighting: we jointly use rule-aggregated difficulty and pairwise gradient cosine conflict scores—estimated periodically with exponential moving average smoothing—to set dynamic task weights λ_j that balance PDE, BC, and IC objectives.

We evaluate HD-TSK-PINN on four PDEs chosen to stress-test distinct aspects of the design: Poisson (baseline accuracy on a well-conditioned problem), Burgers (hierarchical rule allocation to shock fronts), Allen–Cahn (difficulty-driven resampling at moving interfaces), and Navier–Stokes (gradient-conflict weighting for coupled multi-output systems). Section 2 reviews related work. Section 3 presents the problem formulation. Section 4 describes the architecture and all mathematical derivations. Section 5 details the training algorithm. Section 6 reports experiments. Section 7 discusses limitations and extensions. Section 8 concludes.

2. Related Work

2.1 Physics-Informed Neural Networks

Raissi et al. [2] proposed the original PINN framework. Subsequent analysis identified gradient flow pathologies [5] and spectral bias [6] as core obstacles. Wang et al. [5] showed that different PINN loss terms exhibit discrepant convergence rates under the neural tangent kernel (NTK) perspective, motivating adaptive learning rate strategies. Causal PINN training [12] and Fourier feature embeddings [13] further improve time-dependent and high-frequency problems. On the sampling side, residual-based adaptive refinement (RAR) [14] and PINNACLE [15] allocate collocation effort to high-residual regions. PINNacle [16] benchmarked many PINN variants and

highlighted domain decomposition and loss reweighting as the most impactful directions for complex geometry and multi-scale problems.

2.2 Domain Decomposition for PINNs

FBPINNs [7] decompose the domain into overlapping subdomains with local networks and demonstrated the superiority of multilevel decomposition on high-frequency problems. APINN [8] introduces trainable gating functions for soft decomposition with parameter sharing across subdomains, showing that shared parameters and soft partitioning are synergistic. Parallel PINNs via classical Schwarz decomposition [16b] provide complementary numerical analysis perspectives. None of these methods couple the decomposition to region-level PDE difficulty or gradient conflict.

2.3 Region Optimization

RoPINN [4], presented at NeurIPS 2024, extends collocation optimization from isolated scatter points to continuous neighborhood regions. Its theoretical contribution is a generalization error bound showing that region optimization with trust-region calibration reduces error even for PDEs with hidden higher-order constraints. Our region-level difficulty scoring is inspired by the same intuition—that point-level residuals are gameable by the optimizer—but differs in mechanism: we aggregate neighborhood-smoothed residuals by fuzzy rule responsibility, producing a per-rule difficulty signal that drives sampling and rule evolution rather than directly modifying the optimization objective.

2.4 Multi-task Loss Balancing

Self-adaptive PINNs [17] learn scalar loss weights via a min-max formulation. Wang et al. [5] propose NTK-based learning rate annealing. Gradient alignment methods [18] directly compute pairwise gradient cosine similarities to diagnose directional conflict. Our method differs by obtaining task difficulty from rule-level statistics (a fuzzy-structure-derived signal) and combining it with gradient conflict scores, rather than relying on either signal alone.

2.5 Fuzzy Systems and PINNs

FPINNs [10] integrate Gaussian membership function layers and rule layers into PINN architectures for uncertainty-aware forward/inverse solving. CAFPINN [11] adds cross-attention fusion between neural and fuzzy feature branches, reporting accuracy and stability improvements. Our work differs fundamentally in motivation: we use the hierarchical TSK structure as the organizing principle of the entire solver—domain partition, local expert design, difficulty monitoring, and training coordination—rather than as an auxiliary feature enhancer.

3. Problem Formulation

3.1 General PDE Setting

We consider a PDE boundary-initial value problem on spatial domain $\Omega \subset R^d$ and time interval $[0,T]$ (omitted for steady-state problems). Let the full spatiotemporal domain be $Q = \Omega \times (0,T]$. The target field is $u(\xi) \in R^m$, $\xi = (x,t) \in Q$. The governing equation, boundary condition, and initial condition are:

$$F[u](\xi) = s(\xi), \quad \xi \in Q_f \quad (PDE) \quad (1)$$

$$B[u](\xi) = g_b(\xi), \quad \xi \in \Gamma_b \quad (BC) \quad (2)$$

$$I[u](x) = g_i(x), \quad x \in \Omega \quad (IC) \quad (3)$$

where $F[\cdot]$ is the (possibly nonlinear) differential operator, $B[\cdot]$ is the boundary operator, $I[\cdot]$ is the initial operator, and $m = 1$ for scalar PDEs ($m > 1$ for coupled systems such as Navier–Stokes). The neural approximation $u_\theta(\xi)$ is evaluated using automatic differentiation for all required partial derivatives.

3.2 PINN Losses and the Core Challenge

We sample collocation sets $D_f = \{\xi_n^f\}_{n=1}^{N_f}$, $D_b = \{\xi_n^b\}_{n=1}^{N_b}$, $D_i = \{x_n^i\}_{n=1}^{N_i}$ and define three point-wise residuals:

$$r_f(\xi) = F[u_\theta](\xi) - s(\xi), \quad r_b(\xi) = B[u_\theta](\xi) - g_b(\xi), \quad r_i(x) = I[u_\theta](x) - g_i(x) \quad (4)$$

with corresponding mean-squared losses L_f, L_b, L_i. The composite PINN loss is:

$$L = \lambda_f L_f + \lambda_b L_b + \lambda_i L_i \quad (5)$$

The task weights λ_f, λ_b, λ_i are critical hyperparameters. Fixed weights fail because: (a) the PDE difficulty landscape is spatially nonuniform and evolves during training; (b) the three loss terms can produce directional gradient conflicts with respect to shared network parameters. These two facts jointly motivate the design of HD-TSK-PINN.

3.3 Representative PDE Benchmarks

We use four PDEs, each stressing a different aspect of the proposed framework.

(1) Poisson: $-\Delta u(x) = f(x)$ on $\Omega = [0,1]^2$, $u|\partial\Omega = g$. Tests baseline accuracy on a well-conditioned steady problem. Exact solution: $u = \sin(\pi x_1)\sin(\pi x_2)$, $f = 2\pi^2 u$.

(2) Burgers: $u_t + u \cdot u_x = \nu \cdot u_{xx}$ on $(x,t) \in [-1,1]\times[0,1]$, $u(x,0) = -\sin(\pi x)$, $u(\pm 1,t) = 0$, $\nu = 0.01/\pi$. Develops a steep shock front; tests hierarchical rule allocation.

(3) Allen–Cahn: $u_t - \varepsilon \cdot u_{xx} + (1/\varepsilon)(u^3 - u) = 0$ on $(x,t) \in [-1,1]\times[0,1]$, $\varepsilon = 0.001$, periodic BC: $u(-1,t) = u(1,t)$, $u_x(-1,t) = u_x(1,t)$, $u(x,0) = x^2\cos(\pi x)$. Moving interface tests difficulty-driven resampling.

(4) Navier–Stokes (lid-driven cavity): steady incompressible, $(u\cdot\nabla)u - \nu\nabla^2 u + \nabla p = 0$, $\nabla\cdot u = 0$ on $\Omega = [0,1]^2$, $\nu = 0.01$. BC: top wall (y=1): u=1, v=0; all other walls: u=v=0. Multi-output coupling tests gradient-conflict weighting.

4. HD-TSK-PINN: Architecture and Formulation

Figure 1 (to be added) provides a schematic of the full framework. We describe the four components in order.

4.1 Hierarchical TSK Antecedent

4.1.1 Motivation

PDE solutions exhibit multi-scale structure: smooth low-frequency behavior over most of the domain, with local high-frequency features (shocks, phase interfaces, boundary layers) concentrated in narrow regions. A flat set of K rules must allocate its capacity across all scales simultaneously—either too coarse for singular features or too fine to efficiently capture smooth backgrounds. A two-level hierarchy provides a natural inductive bias: coarse rules handle global smooth structure, and fine rules (nested under coarse rules) refine locally difficult subregions. This mirrors the multilevel domain decomposition principle of FBPINNs [7], realized here through differentiable fuzzy membership functions rather than hard geometric boundaries.

Importantly, the antecedent is defined on physical coordinates $\varphi(\xi) \in \mathbb{R}^{d_\varphi}$ rather than latent features, preserving spatial interpretability. After training, the rule centers $c_a^{(c)}$ and $c_{ab}^{(f)}$ can be visualized in the solution domain to reveal what subregions each rule has claimed.

4.1.2 Coarse-Level Rules

The coarse level contains K_c rules. For rule $a \in \{1,\ldots,K_c\}$, the antecedent firing strength uses a diagonal Gaussian (one σ per input dimension, avoiding the positive-definiteness constraint of a full covariance matrix):

$$\mu_a^{(c)}(\xi) = \exp\left(-\Sigma_{k=1}^{d_\varphi} (\varphi_k(\xi) - c_{ak}^{(c)})^2 / (2(\sigma_{ak}^{(c)})^2) \right) \quad (6)$$

where $c_{ak}^{(c)}, \sigma_{ak}^{(c)} \in \mathbb{R}$ are learnable center and width for rule a on dimension k, with $\sigma_{ak}^{(c)} > 0$ enforced via softplus. The normalized coarse responsibility is:

$$\alpha_a(\xi) = \mu_a^{(c)}(\xi) / \left(\Sigma_{a'=1}^{K_c} \mu_{a'}^{(c)}(\xi) + \varepsilon \right), \quad \Sigma_a \alpha_a(\xi) = 1 \quad (7)$$

with $\varepsilon > 0$ for numerical stability. Note that Eq. (7) sums to 1 by construction (partition of unity at the coarse level).

4.1.3 Fine-Level Rules

Each coarse rule a has K_f fine rules nested under it. For fine rule $b \in \{1,\ldots,K_f\}$ under coarse rule a:

$$\mu_{ab}^{(f)}(\xi) = \exp\left(-\Sigma_{k=1}^{d_\varphi} (\varphi_k(\xi) - c_{abk}^{(f)})^2 / (2(\sigma_{abk}^{(f)})^2) \right) \quad (8)$$

$$\beta_{ab}(\xi) = \mu_{ab}^{(f)}(\xi) / \left(\Sigma_{b'=1}^{K_f} \mu_{ab'}^{(f)}(\xi) + \varepsilon \right), \quad \Sigma_b \beta_{ab}(\xi) = 1 \text{ for each } a \quad (9)$$

The joint hierarchical responsibility of rule pair (a,b) is:

$$\gamma_{ab}(\xi) = \alpha_a(\xi) \cdot \beta_{ab}(\xi) \quad (10)$$

Partition of unity: $\Sigma_a \Sigma_b \gamma_{ab}(\xi) = \Sigma_a \alpha_a(\xi) \cdot (\Sigma_b \beta_{ab}(\xi)) = \Sigma_a \alpha_a(\xi) \cdot 1 = 1$ for all ξ.

This hierarchical factorization has a concrete semantic meaning: coarse rule a claims what fraction of 'responsibility' belongs to the a-th global region; fine rule b|a then further divides that region's responsibility among local sub-patches. A high-difficulty fine rule can be split into two (Section 4.3) without disturbing other coarse regions.

4.2 Shared-Backbone Adapter Consequents

4.2.1 Motivation and Design

A naive TSK design assigns a complete MLP to each of the $K_c \times K_f$ rule pairs, giving parameter count $O(K_c K_f \cdot W^2)$ where W is the hidden width. This is prohibitive as the rule count grows (especially under the grow/split mechanism of Section 4.3). Drawing on the parameter-sharing insight of APINN [8] and adapter-based fine-tuning in transfer learning, we decouple global feature extraction from local rule specialization.

A single shared backbone $z(\xi) = \text{MLP_base}(\xi) \in R^{d_z}$ extracts universal PDE features—smooth backgrounds, common gradient patterns, dominant function classes—and is shared across all rules. A global output head produces the base prediction:

$$q_0(\xi) = W_{out} \cdot z(\xi) + b_{out}, \qquad q_0 \in R^m \quad (11)$$

Each rule pair (a,b) has a lightweight adapter $A_{ab}: R^{d_z} \to R^m$ with a single hidden layer of dimension $d_h \ll d_z$:

$$A_{ab}(z) = W_{ab}^{(2)} \cdot \sigma(W_{ab}^{(1)} \cdot z + b_{ab}^{(1)}) + b_{ab}^{(2)} \quad (12)$$

where $W_{ab}^{(1)} \in R^{d_h \times d_z}$, $W_{ab}^{(2)} \in R^{m \times d_h}$. The rule consequent is:

$$q_{ab}(\xi) = q_0(\xi) + A_{ab}(z(\xi)) \quad (13)$$

4.2.2 Final Solution

The HD-TSK-PINN approximation is the responsibility-weighted mixture:

$$u_\theta(\xi) = \Sigma_a \Sigma_b \gamma_{ab}(\xi) \cdot q_{ab}(\xi) \quad (14)$$

Substituting Eq. (13) and using the partition of unity:

$$u_\theta(\xi) = q_0(\xi) + \Sigma_a \Sigma_b \gamma_{ab}(\xi) \cdot A_{ab}(z(\xi)) \quad (15)$$

The first term $q_0(\xi)$ is a globally trained base prediction. The second term is a responsibility-weighted sum of local corrections: region (a,b) contributes its adapter output A_{ab} in proportion to $\gamma_{ab}(\xi)$. This decomposition is semantically clean: the backbone learns the average PDE behavior; each adapter learns a residual correction for its subregion.

4.2.3 Parameter Count

Adapter parameters per rule: $d_h \cdot d_z + m \cdot d_h = d_h(d_z + m)$. For $K_c = 4$, $K_f = 3$, $d_z = 128$, $m = 1$, $d_h = 16$: total $12 \times 16 \times 129 = 24{,}768$ adapter parameters—compared to 12 independent MLPs of width 128 which would need approximately 12× more. Adding one new fine rule (during rule splitting) requires only 2 small weight matrices.

4.3 Region-Level Difficulty Scoring and Rule Evolution

4.3.1 Why Point Residuals Are Insufficient

The obvious approach to measuring rule-region difficulty is to aggregate point-level PDE residuals $|r_f(\xi)|^2$ by rule responsibility. However, this suffers from a fundamental weakness: a single gradient step can drastically reduce the residual at a specific collocation point ξ_n without improving the solution in the surrounding neighborhood. The optimizer exploits the local curvature at ξ_n, effectively 'memorizing' that point while violating the PDE elsewhere. As a result, point residuals are gameable—a rule may appear easy (low averaged point residuals) while its spatial region is actually poorly solved. This is the core insight behind RoPINN [4]: optimizing over neighborhoods rather than points leads to better generalization.

We adopt a related strategy but via a different mechanism: instead of modifying the optimization objective, we compute a neighborhood-smoothed residual for use as a difficulty diagnostic and sampling signal. This keeps the primary optimization clean while providing a robust, difficult-to-game difficulty estimate.

4.3.2 Neighborhood-Smoothed Residual

For each PDE collocation point ξ, define the neighborhood-smoothed residual by averaging over M points sampled uniformly from the ε-ball $B_\varepsilon(\xi)$:

$$\tilde{\ell}_f(\xi) = (1/M) \Sigma_{m=1}^{M} |r_f(\xi + \delta_m)|^2, \qquad \delta_m \sim \mathrm{Uniform}(B_\varepsilon(0)) \quad (16)$$

The perturbations $\{\delta_m\}$ are sampled once per difficulty evaluation cycle and treated as fixed offsets—they require only forward passes through the PDE operator (no additional backward pass). Equation (16) requires the network to satisfy the PDE in an entire neighborhood of ξ, making it robust to the point-memorization phenomenon. For BC and IC points, which lie on lower-dimensional manifolds, we retain point-level residuals to avoid perturbations leaving the boundary manifold:

$$\ell_b(\xi) = |r_b(\xi)|^2, \qquad \ell_i(x) = |r_i(x)|^2 \quad (17)$$

To reduce scale mismatch across tasks, we normalize each point-wise difficulty by an EMA of the task loss:

$$\tilde{\ell}_j^{norm}(\xi) = \tilde{\ell}_j(\xi) \,/\, (\bar{L}_j + \varepsilon), \qquad \bar{L}_j \leftarrow \rho_L \cdot \bar{L}_j + (1-\rho_L) \cdot L_j \quad (18)$$

where $\bar{L}_j$ is initialized to 1 and $\rho_L = 0.99$. This normalization ensures that difficulty scores for PDE, BC, and IC are on a comparable scale regardless of their absolute loss magnitudes.

4.3.3 Rule-Level Difficulty Aggregation

The rule-level difficulty score for task $j \in \{f, b, i\}$ and rule (a,b) is the responsibility-weighted average of normalized point-wise difficulties over the corresponding collocation set D_j:

$$d_{ab,j} = [\, \Sigma_{\xi \in D_j} \gamma_{ab}(\xi) \cdot \tilde{\ell}_j^{norm}(\xi) \,] \,/\, [\, \Sigma_{\xi \in D_j} \gamma_{ab}(\xi) + \varepsilon \,] \quad (19)$$

The denominator $\Sigma_{\xi \in D_j} \gamma_{ab}(\xi) + \varepsilon$ is the support of rule (a,b) on task j, denoted $\pi_{ab,j}$ (before adding ε). The combined rule difficulty is:

$$D_{ab} = d_{ab,f} + \beta_b \cdot d_{ab,b} + \beta_i \cdot d_{ab,i} \quad (20)$$

where $\beta_b, \beta_i \in \mathbb{R}^+$ are balancing coefficients (default 1.0). The global task-level difficulty is a support-normalized weighted average of rule difficulties:

$$d_j = [\, \Sigma_r \bar{\pi}_r \cdot d_{r,j} \,] \,/\, [\, \Sigma_r \bar{\pi}_r \,], \qquad \bar{\pi}_r = (1/N_j) \Sigma_{\xi \in D_j} \gamma_r(\xi) \quad (21)$$

The explicit denominator in Eq. (21) is necessary because $\bar{\pi}_r$ is a per-rule average over a finite set, and $\Sigma_r \bar{\pi}_r \neq 1$ in general (it equals 1 only point-wise, not after averaging). This normalization ensures $d_j \in [\min_r d_{r,j}, \max_r d_{r,j}]$.

4.3.4 Three Uses of D_{ab}

The rule difficulty score D_{ab} drives three complementary mechanisms:

(1) Adaptive PDE Resampling. Every T_s steps, a fraction of PDE collocation points are replaced by new samples. The sampling probability for a candidate point ξ is:

$$p_f(\xi) \propto \Sigma_{a,b} \gamma_{ab}(\xi) \cdot \exp(D_{ab} / \tau_s) \quad (22)$$

This assigns higher probability to points lying in high-difficulty, highly-activated rule regions. The exponential weighting with temperature τ_s controls the sharpness of the resampling distribution.

(2) Rule Splitting (Growing). If a fine rule (a,b) has both high difficulty and sufficient collocation support, it is flagged as a candidate for splitting into two child rules in the next evolution cycle:

$$\text{Split: } D_{ab} > \tau_D \quad \text{AND} \quad \pi_{ab,f} > \tau_{grow} \tag{23}$$

A new rule center is initialized at the centroid of the high-residual subset of the rule's support points. The parent rule's adapter parameters are copied to both children as a warm start. Splitting is conservative: a rule must satisfy Eq. (23) for T_{split} consecutive evaluation cycles before being split.

(3) Rule Pruning. A rule with persistently low difficulty and negligible support is pruned:

$$\text{Prune: } D_{ab} < \tau_{prune} \quad \text{AND} \quad \pi_{ab,f} < \tau_{small} \quad \text{(for } T_{prune} \text{ consecutive cycles)} \tag{24}$$

Pruning removes the adapter $\{W_{ab}^{(1)}, W_{ab}^{(2)}\}$ without affecting the shared backbone. The coarse level is never pruned to ensure full domain coverage.

4.4 Rule-Balance Regularization

Even with the hierarchical initialization (Section 5.1), rules may collapse early if one rule's responsibility dominates all collocation points. We add a lightweight balance regularizer that penalizes deviation from uniform activation at both levels:

$$L_{bal} = \Sigma_{a=1}^{K_c} (\bar{\alpha}_a - 1/K_c)^2 + (1/K_c) \Sigma_a \Sigma_b (\bar{\beta}_{ab} - 1/K_f)^2 \tag{25}$$

where $\bar{\alpha}_a = (1/N) \Sigma_n \alpha_a(\xi_n)$ and $\bar{\beta}_{ab} = (1/N) \Sigma_n \beta_{ab}(\xi_n)$ are the empirical mean activations over all N training points. This regularizer discourages any coarse or fine rule from claiming zero responsibility everywhere, while not forcing strict uniformity—rules are still free to specialize. Its weight λ_{bal} is set to a small value (default 1e-4) and is held fixed throughout training.

4.5 Gradient-Conflict-Aware Task Weighting

4.5.1 The Directional Conflict Problem

Magnitude-based adaptive weighting strategies (e.g., [17]) increase the weight of loss terms with large current magnitude. However, Wang et al. [5] and gradient alignment work [18] identify a more subtle failure: a loss term may have small magnitude yet produce gradients that actively

oppose those of another term—directional conflict. Net cancellation of gradient signal cannot be detected or corrected by magnitude-only strategies.

4.5.2 Periodic Conflict Estimation with EMA

Let $\theta_{sh} \subset \theta$ denote the shared backbone and global head parameters (those computed over by all rules and all tasks). Every T_g training steps, we compute the per-task gradient with respect to θ_{sh}:

$$g_j = \nabla_{\theta_{sh}} L_j(\theta), \quad j \in \{f, b, i\} \tag{26}$$

The conflict score for task j is the mean negative cosine similarity with all other tasks:

$$c_j = (1/2) \Sigma_{k \neq j} \max(0, -\cos(g_j, g_k)), \quad \cos(g_j, g_k) = \langle g_j, g_k \rangle / (\|g_j\| \cdot \|g_k\| + \varepsilon) \tag{27}$$

A positive c_j means task j's gradient direction actively opposes at least one other task. To avoid computing Eq. (27) every step (which would triple training cost), we maintain EMA estimates with decay coefficient β_{ema} (note: this is distinct from ρ_c below, which controls conflict influence on task weights):

$$\hat{c}_j \leftarrow \beta_{ema} \cdot \hat{c}_j + (1 - \beta_{ema}) \cdot c_j, \quad \beta_{ema} = 0.9 \tag{28}$$

Similarly, EMA-smoothed task difficulties:

$$\hat{d}_j \leftarrow \beta_d \cdot \hat{d}_j + (1 - \beta_d) \cdot d_j, \quad \beta_d = 0.9 \tag{29}$$

4.5.3 Joint Difficulty-Conflict Task Weights

The composite score for task j is $v_j = \eta \cdot \hat{d}_j + \rho_c \cdot \hat{c}_j$, where $\eta \geq 0$ weights task difficulty and $\rho_c \geq 0$ weights gradient conflict. The normalized softmax weight is:

$$s_j = \exp(v_j / \tau_w) / \Sigma_{k \in \{f,b,i\}} \exp(v_k / \tau_w), \quad \Sigma_j s_j = 1 \tag{30}$$

We apply a floor $\lambda_{min} > 0$ to prevent complete suppression of any task:

$$\lambda_j = \lambda_{min} + (1 - 3\lambda_{min}) \cdot s_j \tag{31}$$

It follows directly that $\Sigma_j \lambda_j = 3\lambda_{min} + (1-3\lambda_{min}) \cdot \Sigma_j s_j = 1$, so Eq. (31) gives a proper probability distribution for all $\lambda_{min} \in (0, 1/3)$. Default settings: $\lambda_{min} = 0.05$, $\eta = 1.0$, $\rho_c = 0.5$, $\tau_w = 1.0$.

The semantics are explicit: $\hat{d}_j$ answers 'Is this task hard to satisfy in the current rule regions?' and $\hat{c}_j$ answers 'Is this task's gradient pulling against the others?' A task that is simultaneously difficult and conflicting receives the largest λ_j. Setting $\rho_c = 0$ recovers a difficulty-only strategy (ablated in Section 6); setting $\eta = 0$ recovers a conflict-only strategy.

The total training loss is:

$$L_{total} = \lambda_f L_f + \lambda_b L_b + \lambda_i L_i + \lambda_{bal} L_{bal} \quad (32)$$

5. Training Algorithm

Algorithm 1 summarizes the complete training procedure. We describe initialization and complexity below.

Algorithm 1: HD-TSK-PINN Training

Phase	Step	Description
Init	1	K-means on domain sample → coarse centers $\{c_a^{(c)}\}$; perturb → fine centers $\{c_{ab}^{(f)}\}$; random init backbone, global head, adapters; set EMA variables $\bar{L}_j=1$, $\hat{d}_j=0$, $\hat{c}_j=0$.
Main loop	2	Sample D_f, D_b, D_i if not yet initialized.
	3	Forward: compute $\gamma_{ab}(\xi)$ via Eqs. (6)-(10); compute $z(\xi)$, $q_0(\xi)$, $A_{ab}(z(\xi))$, $u_\theta(\xi)$ via Eqs. (11)-(15).
	4	Losses: compute L_f, L_b, L_i, L_{bal} via Eqs. (4),(25).
	5 (every T_g steps)	Gradient conflict: compute $g_j = \nabla_{\theta_{sh}} L_j$; update $\hat{c}_j$ via Eq. (28).
	6 (every T_{diff} steps)	Difficulty: compute $\tilde{\ell}_f(\xi)$ via Eq. (16); compute $d_{ab,j}$, D_{ab} via Eqs. (18)-(20); update d_j, $\hat{d}_j$ via Eqs. (21),(29).
	7	Task weights: update λ_j via Eqs. (30)-(31); form L_{total} via Eq. (32).
	8	Backward: compute $\nabla_\theta L_{total}$; update θ via Adam.
Evolution	9 (every T_s steps)	Resample fraction of D_f via Eq. (22).
	10 (every T_{ev} steps)	Rule evolution: check split condition Eq. (23); check prune condition Eq. (24); apply grow/prune with warm-start adapter init.

5.1 Initialization

Coarse rule centers $\{c_a^{(c)}\}$ are initialized via k-means ($k = K_c$) on a random subsample of the collocation domain, ensuring coverage from training start. Fine centers $\{c_{ab}^{(f)}\}$ are initialized as small random perturbations of their parent coarse center $c_a^{(c)}$, scaled by the coarse width $\sigma_a^{(c)}/2$. Widths $\sigma_{ak}^{(c)}$, $\sigma_{abk}^{(f)}$ are initialized to $1/(2K_c)$ and $1/(2K_f)$

of the domain range per dimension, respectively. This warm start avoids the degenerate configuration where all responsibility collapses to one rule.

5.2 Computational Cost Analysis

The dominant cost is the backbone MLP evaluation and AD-based PDE residual computation, identical to standard PINN: $O(N_f \cdot C_{MLP})$. Adapter passes add $O(N_f \cdot K_c K_f \cdot d_h d_z)$, which is small for $d_h \ll d_z$. Neighborhood sampling for difficulty (Eq. 16): M extra forward passes per point per T_{diff} steps, with no backward pass (perturbations are fixed offsets). Gradient conflict (Eq. 26): 3 extra backward passes every $T_g = 10$ steps, amortized to 0.3 extra backward passes per step. Overall overhead relative to vanilla PINN is modest and dominated by the 3 periodic backward passes for conflict estimation.

6. Experiments

6.1 Implementation

HD-TSK-PINN is implemented in PyTorch. Shared backbone: 5-layer MLP, 128 neurons, tanh activation. Adapter: $d_h = 16$. Initial rules: $K_c = 4$ coarse, $K_f = 3$ fine (12 total). Optimizer: Adam (lr = 1e-3) for 50,000 steps, then L-BFGS for 2,000 steps. $T_g = 10$, $T_{diff} = 50$, $T_s = 1{,}000$, $T_{ev} = 5{,}000$. Neighborhood: $\varepsilon = 0.05$, $M = 4$. Rule evolution thresholds: $\tau_D = 0.6$, $\tau_{grow} = 0.1 \cdot N_f/K_{init}$, $\tau_{prune} = 0.05$, $\tau_{small} = 0.01 \cdot N_f/K_{init}$, $T_{split} = T_{prune} = 3$ cycles. Task weight hyperparameters: $\lambda_{min} = 0.05$, $\eta = 1.0$, $\rho_c = 0.5$, $\tau_w = 1.0$. All results averaged over 5 random seeds.

6.2 Baselines

We compare against: (1) PINN [2]—vanilla equal weights; (2) SA-PINN [17]—self-adaptive loss weights; (3) APINN [8]—soft domain decomposition with gating and parameter sharing; (4) RoPINN [4]—region optimization applied to standard PINN; (5) FPINNs [10]—fuzzy membership + rule layers for feature fusion; and three ablations of our model: (6) NoHier—flat rules ($K_c \times K_f$ rules, no hierarchy); (7) NoNbhd—point-level residual difficulty instead of neighborhood-smoothed; (8) NoConflict—difficulty-only weighting ($\rho_c = 0$); (9) NoEvolve—static rules, no grow/split/prune.

6.3 Results

Method	Poisson L^2↓	Burgers L^2↓	Allen-Cahn L^2↓	NS Vel L^2↓	NS Div↓
PINN [2]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

SA-PINN [17]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
APINN [8]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
RoPINN [4]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
FPINNs [10]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
HD-TSK-PINN (ours)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

Table 1: Relative L^2 error (and divergence error for NS) across all methods. [TBD] = to be filled from experiments.

6.4 Ablation Studies

Ablation	Variant	Change from Full Model	Burgers L^2↓	Allen-Cahn L^2↓
A1: Hierarchy	NoHier	Flat $K_c \times K_f$ rules	[TBD]	[TBD]
	HD-TSK-PINN	Full model	[TBD]	[TBD]
A2: Nbhd Residual	NoNbhd	Point residual for difficulty	[TBD]	[TBD]
	HD-TSK-PINN	Neighborhood residual (Eq.16)	[TBD]	[TBD]
A3: Conflict Weight	NoConflict	$\rho_c = 0$ (difficulty only)	[TBD]	[TBD]
	HD-TSK-PINN	Full difficulty + conflict	[TBD]	[TBD]
A4: Rule Evolution	NoEvolve	Static rules	[TBD]	[TBD]
	HD-TSK-PINN	Grow/split/prune enabled	[TBD]	[TBD]

Table 2: Ablation results. Each row pair isolates the contribution of one component. [TBD] = to be filled.

6.5 Analysis per PDE

Poisson verifies that the added architectural complexity does not degrade performance on a well-conditioned problem. Burgers tests whether hierarchical rules (A1) allocate fine rules to the shock front region—rule activation maps $\gamma_{ab}(\xi)$ are visualized post-training to confirm this qualitative behavior. Allen–Cahn tests whether difficulty-driven resampling (Eq. 22) and

neighborhood residual difficulty (A2) concentrate collocation points near the moving interface. Navier–Stokes tests whether gradient conflict weighting (A3) prevents the PDE momentum loss from suppressing the divergence constraint ($u_x + v_y = 0$) during early training.

7. Discussion

7.1 Interpretability

A key advantage of the TSK structure is that rule centers $\{c_a^{(c)}, c_{ab}^{(f)}\}$ and responsibilities $\{\gamma_{ab}(\xi)\}$ can be visualized directly in the physical domain. After training on Burgers, coarse rules are expected to align with physically meaningful regions (upstream, downstream, shock transition zone) without explicit supervision—the fuzzy rule centers learn this partition purely from PDE residual signals. Fine rules within the transition zone spread to cover the temporal evolution of the shock position. This emergent interpretability is absent in standard PINNs, gated networks (APINN), or attention-based fuzzy hybrids (CAFPINN).

7.2 Relation to RoPINN

Our neighborhood residual (Eq. 16) is inspired by RoPINN's [4] theoretical analysis but differs in mechanism. RoPINN modifies the optimization objective to optimize over a region; we leave the optimization objective unchanged and use neighborhood residuals only as a diagnostic signal for difficulty scoring and resampling. This is a deliberate design choice: modifying the PINN loss function with high-order neighborhood terms introduces additional AD complexity, while our approach has negligible extra cost (M forward passes with fixed offsets). The claim is not that our approach matches RoPINN's theoretical guarantees for generalization error; rather, it is that fuzzy rule aggregation of neighborhood residuals provides a robust, interpretable, and low-cost alternative to region-optimization for the specific purpose of difficulty-guided training coordination.

7.3 Limitations

Three limitations merit mention. First, K_c and K_f are hyperparameters requiring problem knowledge for initialization; fully automated rule count determination (e.g., via BIC or Bayesian optimization) is left for future work. Second, the diagonal Gaussian membership functions may be insufficient for highly anisotropic regions; full or factored covariance formulations could improve coverage at the cost of positive-definiteness constraints. Third, all experiments are forward-problem benchmarks; extension to inverse problems requires integrating observation losses and parameter identification uncertainty into the difficulty and conflict framework.

7.4 Extensions

The HD-TSK-PINN framework extends naturally in several directions. For inverse problems, an additional data-fitting loss L_data can be added to the task set T, with its difficulty and conflict score computed identically to L_f. For operator learning, the hierarchical rule structure could be applied over a parameter space rather than a single-instance domain. The rule evolution mechanism (grow/split/prune) could also be enriched with rule merging, where two low-support rules covering the same region are collapsed to reduce model complexity after initial training.

8. Conclusion

We proposed HD-TSK-PINN, a framework that repositions the TSK fuzzy inference system from a peripheral feature-fusion layer to the organizational backbone of physics-informed neural network training. Four tightly coupled contributions address complementary failure modes of standard PINNs: a two-level hierarchical antecedent for multi-scale domain decomposition; a shared-backbone plus rule-adapter consequent for parameter-efficient local modeling; region-level difficulty scoring via neighborhood Monte Carlo residuals that drives adaptive resampling and rule evolution; and gradient-conflict-aware task weighting that jointly accounts for task difficulty and inter-task gradient alignment. An additional rule-balance regularizer prevents early rule collapse. On four canonical PDE benchmarks—Poisson, Burgers, Allen–Cahn, and Navier–Stokes—HD-TSK-PINN is designed to outperform vanilla PINNs and recent fuzzy-PINN and domain-decomposition baselines, with ablation studies confirming the contribution of each component. The core conceptual contribution is a principled way to let structured prior knowledge (fuzzy rule partitioning) and training dynamics (difficulty, gradient conflict) inform each other in a unified framework.

References

[1] Quarteroni, A., Manzoni, A., & Negri, F. (2016). Reduced basis methods for partial differential equations. Springer.

[2] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks. Journal of Computational Physics, 378, 686–707.

[3] Cai, S., Mao, Z., Wang, Z., Yin, M., & Karniadakis, G. E. (2022). Physics-informed neural networks for fluid mechanics: A review. Acta Mechanica Sinica, 37, 1727–1738.

[4] Wu, H., Luo, H., Ma, Y., Wang, J., & Long, M. (2024). RoPINN: Region Optimized Physics-Informed Neural Networks. NeurIPS 2024.

[5] Wang, S., Teng, Y., & Perdikaris, P. (2021). Understanding and mitigating gradient flow pathologies in physics-informed neural networks. SIAM Journal on Scientific Computing, 43(5), A3055–A3081.

[6] Wang, S., Yu, X., & Perdikaris, P. (2022). When and why PINNs fail to train: A neural tangent kernel perspective. Journal of Computational Physics, 449, 110768.

[7] Moseley, B., Markham, A., & Nissen-Meyer, T. (2023). Finite basis physics-informed neural networks (FBPINNs). Machine Learning: Science and Technology, 4, 025002.

[8] Hu, Z., Shi, Z., Karniadakis, G. E., & Kawaguchi, K. (2024). Augmented physics-informed neural networks (APINNs). Engineering Applications of Artificial Intelligence, 128, 107485.

[9] Takagi, T., & Sugeno, M. (1985). Fuzzy identification of systems and its applications to modeling and control. IEEE Transactions on Systems, Man, and Cybernetics, SMC-15(1), 116–132.

[10] Wu, W., Duan, S., Sun, Y., Yu, Y., Liu, D., & Peng, D. (2025). Deep fuzzy physics-informed neural networks for forward and inverse PDE problems. Neural Networks, 181, 106750.

[11] Huang, C., Jing, X., Hou, B., & Yu, B. (2026). Physics-informed neural networks enhanced by cross-attention fuzzy logic for solving nonlinear PDEs. Neurocomputing, 670.

[12] Wang, S., Sankaran, S., & Perdikaris, P. (2024). Respecting causality for training physics-informed neural networks. Computer Methods in Applied Mechanics and Engineering, 421, 116813.

[13] Wang, S., Wang, H., & Perdikaris, P. (2021). On the eigenvector bias of Fourier feature networks. Computer Methods in Applied Mechanics and Engineering, 384, 113938.

[14] Lu, L., Meng, X., Mao, Z., & Karniadakis, G. E. (2021). DeepXDE. SIAM Review, 63(1), 208–228.

[15] Lau, G. K. R. et al. (2024). PINNACLE: PINN adaptive collocation and experimental points selection. ICLR 2024.

[16] Hao, Z. et al. (2023). PINNacle: A comprehensive benchmark of physics-informed neural networks for solving PDEs. arXiv:2306.08827.

[16b] Shukla, K., Jagtap, A. D., & Karniadakis, G. E. (2021). Parallel PINNs via domain decomposition. Journal of Computational Physics, 447, 110683.

[17] Xiang, Z., Peng, W., Liu, X., & Yao, W. (2022). Self-adaptive loss balanced physics-informed neural networks. Neurocomputing, 496, 11–34.

[18] Hwang, Y., & Lim, D. Y. (2024). Dual cone gradient descent for training physics-informed neural networks. arXiv:2409.00401.